

AMORO-RAG : Adaptive Multi-Objective Reinforcement Optimization for RAG

Gwanyeop Kim, Wonkee Lee, Seunghoon Na

Abstract

This paper addresses a multi-objective reinforcement learning (RL) optimization problem for Retrieval-Augmented Generation (RAG) systems, jointly considering accuracy, faithfulness, cost, and latency. While recent RAG-RL studies incorporate multi-objective rewards, most rely on fixed reward weights or offline search procedures, causing the optimization to effectively behave as a single-objective process in practical deployment settings. To overcome this limitation, we introduce the concept of instance-conditioned scalarization, which learns reward weights at the instance level based on the contextual characteristics of each input query. Building on this idea, we propose AMORO-RAG, a framework that integrates a lightweight weight prediction network (WeightNet) with reinforcement learning using Proximal Policy Optimization (PPO) to dynamically generate reward weight vectors from query features. Experiments on public QA benchmarks demonstrate that AMORO-RAG achieves a more favorable performance–efficiency trade-off compared to fixed-weight baselines and offline Pareto-based optimization methods.

CCS Concepts

• Information systems → Information retrieval; • Computing methodologies → Machine learning.

Keywords

retrieval-augmented generation, reinforcement learning for information retrieval, multi-objective optimization, instance-conditioned optimization, query-aware modeling

ACM Reference Format:

Gwanyeop Kim, Wonkee Lee, Seunghoon Na, . 2026. AMORO-RAG : Adaptive Multi-Objective Reinforcement Optimization for RAG. In . ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

In practical deployment settings of Retrieval-Augmented Generation (RAG) systems [18], a single model is often required to serve highly heterogeneous queries under strict resource and reliability constraints. User requests can vary substantially in terms of length, reasoning complexity, urgency, and tolerance to latency or cost, even within the same application domain. For example, a short factual query may prioritize fast and inexpensive responses, whereas

a multi-hop analytical question may require higher accuracy and stronger evidence grounding, even at the expense of increased computation. These variations make it fundamentally challenging to identify a single operating point that is optimal for all queries in real-world RAG deployments [2, 4, 11, 12, 14, 34].

As a result, trade-offs among accuracy, cost, latency, and faithfulness have become a central concern in recent RAG research [3, 6, 8, 11]. To address these competing objectives, several reinforcement learning (RL)-based RAG approaches define multi-objective reward functions that jointly consider answer correctness, computational efficiency, and hallucination avoidance [13, 20, 38]. However, in most existing methods, the relative importance of reward components is either fixed a priori or determined through offline optimization, resulting in a single reward configuration that is applied uniformly across all queries, regardless of their contextual characteristics.

This static treatment of reward weights introduces a fundamental limitation. Although such methods are formulated as multi-objective optimization problems, their behavior at inference time often collapses to that of single-objective optimization, implicitly enforcing the same preference structure for all inputs. Offline Pareto-front exploration or time-based annealing strategies [3, 20, 22, 36] can identify favorable global trade-offs, but they cannot adapt reward priorities at the level of individual queries. Consequently, these approaches are unable to reflect query-specific requirements, such as prioritizing efficiency for simple questions or emphasizing faithfulness for complex, high-risk queries.

In this work, we argue that the core challenge lies not in how to define additional reward components, but in how to adaptively determine their relative importance for each query instance. To this end, we introduce the concept of *instance-conditioned reward scalarization*, in which the aggregation of multiple objectives is treated as a learnable, context-dependent function rather than a fixed design choice [22, 27]. Building on this principle, we propose AMORO-RAG, a reinforcement learning framework that predicts query-specific reward weights from input features and jointly optimizes both the generation policy and the reward weighting mechanism. By enabling instance-level adaptation of reward priorities, AMORO-RAG allows the model to dynamically select different trade-offs across accuracy, cost, latency, and faithfulness, leading to more flexible and deployment-oriented optimization of RAG systems.

To summarize, our main contributions are as follows:

- We formulate *instance-conditioned reward scalarization* for Retrieval-Augmented Generation, reframing multi-objective RAG optimization as a problem of learning query-specific preference structures rather than selecting a single fixed trade-off.
- We propose AMORO-RAG, a reinforcement learning framework that jointly optimizes the generation policy and

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or to publish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

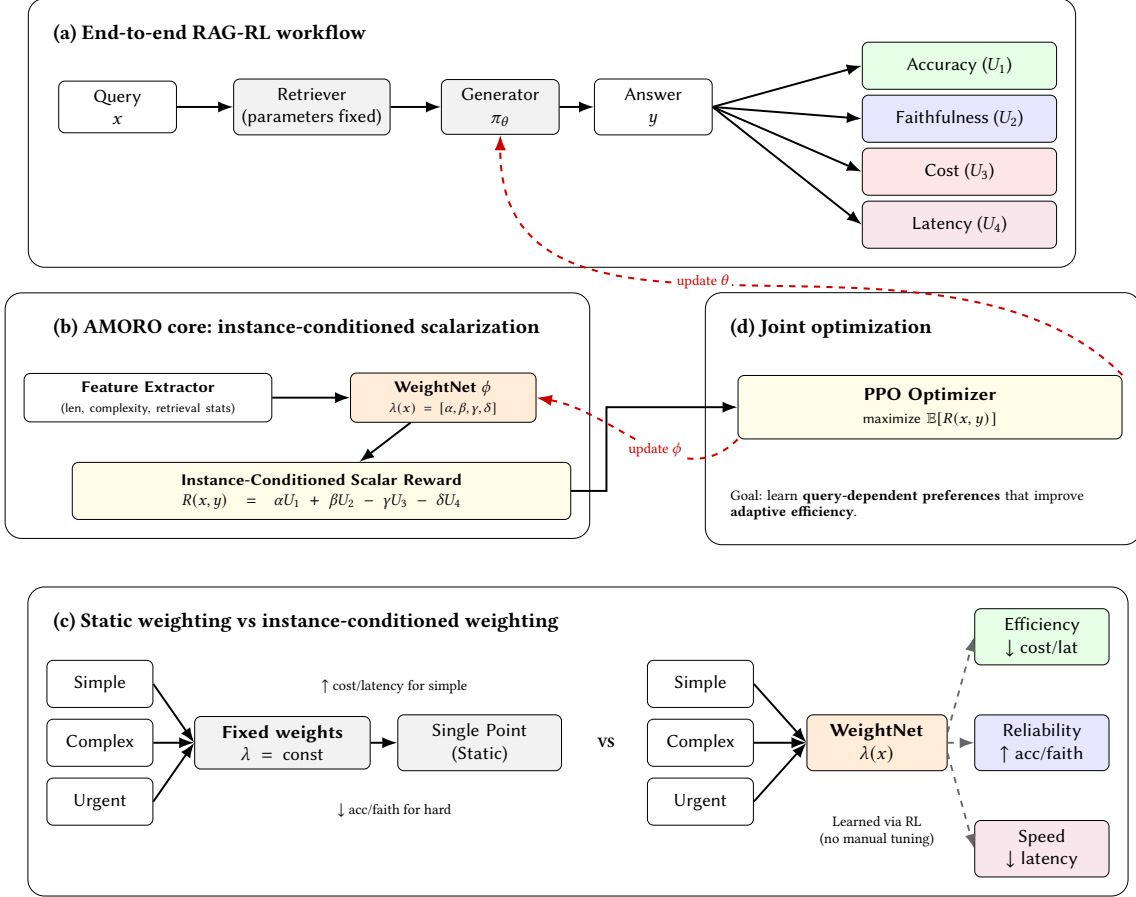


Figure 1: Overview of the proposed AMORO-RAG framework. Given a query, the RAG pipeline generates an answer evaluated by multiple objectives (accuracy, faithfulness, cost, latency). A lightweight WeightNet predicts query-dependent reward weights to form an instance-conditioned scalar reward. Unlike static weighting, AMORO-RAG adapts trade-offs per query, and jointly optimizes the generator and WeightNet via PPO.

a learnable reward weighting mechanism, enabling adaptive trade-offs among accuracy, cost, latency, and faithfulness without offline search or manual tuning.

- We propose to use WeightNet [21], a lightweight prediction module that infers reward weights from query-level features, allowing the reward aggregation function itself to adapt at the instance level during training and inference.
- Extensive experiments on HotpotQA [37], NQ [17] and FEVER [31] demonstrate that AMORO-RAG achieves improved Pareto efficiency compared to static baselines, reducing cost and latency for simpler queries while preserving accuracy and faithfulness for more complex ones.

2 Related Works

Related studies relevant to this work can be broadly categorized into three lines of research.

The first line concerns multi-objective optimization for RAG and LLM-based systems. Prior work in this area primarily explores

offline hyperparameter optimization or Pareto front-based search to balance competing objectives such as accuracy, cost, and latency [3]. While these approaches explicitly acknowledge multi-objective trade-offs, their optimization is performed at the configuration level and remains static after training. Consequently, a single fixed setting is applied regardless of variations in query difficulty or contextual requirements, preventing instance-level adaptation.

The second line of work applies reinforcement learning to RAG systems and reasoning processes, often defining multiple reward components to guide optimization [20, 33, 38]. However, in most cases, reward weights are specified as fixed constants, or adjusted globally across training for stability, rather than being conditioned on the input context. As a result, the relative importance of objectives changes only as a function of the training stage, instead of being tailored to the characteristics of individual queries.

The third line includes studies on adaptive policy selection and SLA-driven system control for RAG pipelines [4, 12]. These approaches adapt execution strategies or policies based on query difficulty, environmental conditions, or system constraints. However, their adaptation operates at the level of pipeline orchestration

or policy selection, while the aggregation of multiple objectives into a scalar reward is typically fixed and not treated as a learnable, query-conditioned component.

Despite their differences, these prior works share a common recognition of the importance of multi-objective optimization in RAG systems. However, they fundamentally differ from our approach in how reward weights are handled. Multi-objective HPO and Pareto-based methods search for optimal trade-offs offline and apply fixed configurations at inference time [3], while RL-based RAG methods rely on predefined or temporally scheduled reward weights [38]. In contrast, our work defines reward weights as functions of query-level input features and incorporates them directly into the reinforcement learning objective. Rather than focusing on how to evaluate multi-objective rewards or which configuration to select, we address the question of which objectives should be emphasized for each query, allowing the model to learn instance-specific reward scalarization. This instance-conditioned formulation is not explicitly modeled in prior approaches based on fixed-weight configurations, offline optimization, or time-based adjustment strategies.

3 Method

3.1 Problem Formulation

We study the problem of evaluating and optimizing the quality of generated responses y for a given input query x in a Retrieval-Augmented Generation (RAG) setting from a multi-objective perspective. Each generated response is assessed using four heterogeneous reward components[30]:

- **Accuracy:** The degree of correctness or task performance with respect to the ground truth,
- **Faithfulness:** The extent to which the generated response is supported by the retrieved evidence [6],
- **Cost:** The number of tokens consumed during generation [4],
- **Latency:** The time required to produce the response[30].

In conventional RAG-RL approaches, these reward components are aggregated into a single scalar reward via a linear combination with predefined, fixed weights [19, 27]. Under this formulation, reinforcement learning optimizes the policy according to a single, static preference structure, regardless of variations in query characteristics. As a result, query-specific requirements—such as differences in difficulty, urgency, or reliability demands—are not reflected in the optimization process.

To overcome this limitation, we define reward weights not as fixed constants, but as functions of the input query x . In other words, we assume that each query induces a distinct multi-objective preference structure. The scalar reward optimized by reinforcement learning is therefore defined as:

$$R(x, y) = \alpha(x) \cdot \text{Acc}(y) + \beta(x) \cdot \text{Faithfulness}(y) - \gamma(x) \cdot \text{Cost}(y) - \delta(x) \cdot \text{Latency}(y) \quad (1)$$

, where

$$\lambda(x) = [\alpha(x), \beta(x), \gamma(x), \delta(x)] \quad (2)$$

denotes the reward weight vector conditioned on the input query x . The central idea of this formulation is to enable reinforcement

learning to jointly learn not only the response generation policy, but also the relative importance of different reward components on a per-query basis.

3.2 Reward Components and Evaluation Metrics

This section formally defines the individual reward components used in our framework. we compute four reward signals capturing accuracy, faithfulness, cost, and latency.

Accuracy. Accuracy measures task performance and is defined differently depending on the dataset. For question answering datasets (HOTPOTQA [37] and NATURAL QUESTIONS [17]), we compute both Exact Match (EM) and token-level F1 between the predicted answer and the ground-truth answer, and define accuracy as $\max(\text{EM}, \text{F1})$. For fact verification (FEVER [31]), where outputs are short categorical labels (SUPPORTS, REFUTES, NOT ENOUGH INFO), accuracy is defined as label-level exact match.

Faithfulness. Faithfulness reflects the extent to which a generated response is grounded in the retrieved evidence. For question answering datasets, we compute a token overlap ratio between the predicted answer and the retrieved contexts. Specifically, let $\mathcal{T}(\hat{y})$ denote the set of non-stopword tokens in the predicted answer and $\mathcal{T}(C)$ the set of tokens in the retrieved contexts. Faithfulness is defined as:

$$\text{Faithfulness}(x) = \frac{|\mathcal{T}(\hat{y}) \cap \mathcal{T}(C)|}{|\mathcal{T}(\hat{y})|}, \quad (3)$$

yielding a continuous score in $[0, 1]$. For FEVER, where answers are short labels and overlap with evidence is not meaningful, we instead measure how well the claim is covered by the retrieved contexts. In this case, faithfulness is computed as the fraction of claim tokens appearing in the retrieved evidence. Concretely, we compute faithfulness using the claim text q in place of the generated response, i.e., by replacing $T(\hat{y})$ with $T(q)$ in Eq. (3). We emphasize that faithfulness is treated as a task-specific proxy signal rather than an absolute measure of factual correctness, and is used consistently across systems to study relative adaptation behavior.

Cost. Cost serves as a proxy for computational expense and is defined as the total number of tokens consumed during retrieval and generation. Cost is defined as:

$$\text{Cost}(x) = \text{clip}\left(\frac{\text{Tokens}(x)}{C}, 0, 1\right) \quad (4)$$

To ensure comparability with other reward components, we normalize this quantity to the range $[0, 1]$ using a fixed upper bound and apply clipping.

Latency. Latency represents end-to-end response time, measured in seconds, including retrieval, model inference, and system overhead. Latency is defined as:

$$\text{Latency}(x) = \text{clip}\left(\frac{\text{Time}(x)}{L}, 0, 1\right) \quad (5)$$

Similar to cost, latency is normalized to $[0, 1]$ using a fixed upper bound and clipped accordingly.

3.3 Instance-Conditioned Reward Scalarization

The central concept of this work is instance-conditioned reward scalarization. Unlike conventional scalarization methods that combine multiple objectives into a single scalar reward using a uniform rule for all inputs [27], instance-conditioned scalarization applies a query-specific aggregation strategy, allowing the scalar reward function to vary across individual query instances.

In traditional static scalarization, the reward weight vector λ is globally fixed and remains unchanged throughout both training and inference. As a result, the model consistently optimizes the same trade-off among accuracy, cost, latency, and faithfulness, regardless of the nature of the input query. This forces the system to pursue a single, uniform preference structure, independent of whether the query is simple or complex, time-sensitive or reliability-critical.

In contrast, our approach is based on the observation that the optimal trade-off among objectives can differ substantially across queries [19, 22, 36]. Such differences are often reflected in observable input features, including query length, structural complexity, and the distribution of retrieval quality scores.

Instance-conditioned scalarization leverages these input features to determine reward weights dynamically, thereby incorporating the question of how to combine multiple objectives directly into the learning process. While existing methods assume a static scalar reward function, our formulation defines a dynamic optimization problem in which a distinct scalar reward function is generated for each query instance. This shift fundamentally reframes reward aggregation from a fixed design choice into a learnable, context-dependent component of reinforcement learning.

3.4 WeightNet Architecture

To implement query-conditioned reward weights $\lambda(x)$, we introduce an auxiliary neural module termed WeightNet, inspired by hypernetwork designs [9, 21]. WeightNet is a lightweight multi-layer perceptron (MLP) that takes as input a feature vector extracted from the query x and outputs a weight vector representing the relative importance of each reward component. The output is normalized using a softmax function, ensuring non-negativity and enforcing the constraint that all weights sum to one.

The role of WeightNet extends beyond simple weight prediction. By dynamically combining heterogeneous reward components—such as accuracy, cost, latency, and faithfulness, which differ substantially in scale and distribution—WeightNet helps mitigate reward scale mismatch[32]. In addition, it reduces training instability by preventing reward dominance, where a single objective overwhelms others during optimization.

However, making reward weights adaptive via a prediction network from the beginning of training can introduce instability during early training [10]. To address this issue, we adopt a staged training strategy, described below.

Two-Stage Training Protocol for Stability. Jointly training WeightNet and the generator from scratch can lead to lack of a reliable signal at early training stages and unstable optimization due to their mutual dependence through the reward function. To alleviate this issue, we propose a two-stage training protocol.

Phase A: Fixed-Weight Warm-up (Stabilization).

During the first 10 training epochs, WeightNet is frozen, and the RAG policy π_θ is trained using Proximal Policy Optimization (PPO) with predefined, fixed reward weights. This phase serves as an anchoring stage, allowing the model to acquire basic question-answering capabilities and establish a stable understanding of contextual grounding and faithfulness.

Phase B: Adaptive Weight Fine-tuning (Optimization).

Once the policy has stabilized, WeightNet ϕ is activated to produce dynamic reward weights $\lambda(x)$ conditioned on query features. In this phase, both the generator and WeightNet are jointly updated to maximize the scalar reward $R(x, y; \lambda(x))$. This phase also aligns with PPO's trust-region style updates, providing additional stability during early training.

Although WeightNet is trained without explicit supervision, it learns implicitly through PPO updates by observing which weight combinations lead to higher cumulative returns. For example, the model may learn to prioritize cost efficiency for simple queries or emphasize accuracy for complex queries.

3.5 WeightNet Input Features

The input to WeightNet consists of five features:

Query length. The total number of tokens in the query, normalized to $[0, 1]$. Longer queries often indicate increased reasoning complexity or multi-hop information needs.

Content word count. The number of non-stopword tokens in the query, normalized to $[0, 1]$. This feature captures semantic density and helps distinguish simple factoid queries from more descriptive or compositional questions.

Proper noun or numeric entity indicator. A binary feature indicating whether the query contains proper nouns or numeric entities. Such entities are often associated with higher retrieval dependence and increased risk of hallucination, motivating stronger emphasis on grounding-related objectives.

Heuristic difficulty score. A coarse difficulty indicator taking values in $\{0, 0.5, 1\}$, derived from query length. This feature provides an approximate notion of query difficulty and helps stabilize early-stage reinforcement learning by introducing a weak prior over query complexity.

Corpus size. The size of the retrieval corpus, normalized to $[0, 1]$. Larger corpora typically increase retrieval cost and latency, and this feature allows WeightNet to account for differences in retrieval environments.

Together, these features provide lightweight yet informative signals for conditioning reward weights on query characteristics, enabling WeightNet to adapt trade-offs without relying on dataset-specific heuristics or manual tuning.

3.6 Reinforcement Learning Optimization

The objective of AMORO-RAG is naturally formulated as a reinforcement learning problem in which each generated response is evaluated using a continuous-valued scalar reward. Since reward weights are conditioned on the input query, both the distribution and the scale of the reward vary dynamically throughout training.

Table 1: Task performance per systems. Accuracy is token-level F1. Faithfulness is a dataset-specific proxy score (not directly comparable across datasets). Cost is measured by the number of LLM-generated tokens per query. Latency denotes the end-to-end wall-clock time per query in seconds.

Dataset	System	Accuracy (F1)	Faithfulness	Cost (Tokens)	Latency (s)
HotpotQA	AMORO-RAG	0.139	0.395	3542	16.50
	Fixed-Weight	0.136	0.399	3631	16.56
	Heuristic-Adaptive	0.147	0.398	3652	16.61
	SingleObj-Acc	0.152	0.412	3955	18.55
NQ	AMORO-RAG	0.184	0.376	1566	6.78
	Fixed-Weight	0.185	0.396	1858	7.30
	SingleObj-Acc	0.176	0.425	2633	9.39
FEVER	AMORO-RAG	0.316	0.736	1556	5.12
	Fixed-Weight	0.325	0.745	1872	5.48
	SingleObj-Acc	0.331	0.777	2667	7.00

Under this setting, we adopt Proximal Policy Optimization (PPO) [29] as the optimization algorithm. PPO is well suited for directly maximizing continuous scalar rewards and provides stable learning by enforcing trust-region updates, which limits the magnitude of policy updates[28]. This property is particularly important in our framework, where both the generator policy and WeightNet are jointly updated and the reward function itself evolves during training. By constraining abrupt policy shifts, PPO enables stable optimization even under non-stationary reward distributions induced by instance-conditioned weighting.

In contrast, preference-based learning methods such as Direct Preference Optimization (DPO) [25] rely on relative comparisons between responses and do not explicitly optimize a scalar reward signal. As a result, it is difficult to directly incorporate the continuous rewards computed via query-conditioned weights into the learning objective. Moreover, integrating the learning of reward weights into policy optimization is less straightforward under a purely preference-based formulation. Additionally, learning instance-conditioned reward weights requires backpropagating through continuously valued rewards, which is not directly supported by pairwise preference formulations.

For these reasons, we conclude that PPO is a natural and effective choice of reinforcement learning method for the AMORO-RAG framework, as it naturally supports continuous reward optimization and accommodates the joint learning of policies and instance-conditioned reward weights.

4 Experiments

4.1 Experimental Setup

We evaluate AMORO-RAG on three public benchmarks with distinct query characteristics and constraints: HotpotQA (multi-hop QA), Natural Questions (factoid QA), and FEVER (fact verification). We follow the standard dataset formats and construct subsets of equal size for all benchmarks. Specifically, we use 1,000 query instances for each dataset (HotpotQA, Natural Questions, and FEVER), sampled from the official data splits. All models are trained and evaluated on the same fixed subsets to ensure fair comparison and

to control computational cost. Each system is evaluated under a common metric space that measures (i) task performance, (ii) evidence grounding (faithfulness proxy), and (iii) efficiency. We report accuracy, faithfulness, cost, and latency for all systems to characterize their multi-objective trade-offs. Beyond task-level performance, our primary focus is on understanding how AMORO-RAG adapts reward weighting behavior across queries, which we analyze in detail in subsequent sections.

We compare AMORO-RAG against (1) Fixed-Weight, which optimizes a static linear scalarization shared across all queries. Specifically, the static weight vector is set to $\lambda = [0.4, 0.3, 0.15, 0.15]$ for accuracy, faithfulness, cost, and latency, respectively. These values reflect a balanced but accuracy-dominant preference, and are chosen to approximate commonly used fixed trade-offs in prior multi-objective RAG settings. And (2) Heuristic-Adaptive (HotpotQA only), a rule-based strategy that adjusts weights based on coarse query heuristics. Heuristic-Adaptive is evaluated only on HotpotQA, where coarse query heuristics such as query length and structure are more indicative of reasoning complexity. For NQ and FEVER, which predominantly consist of short factoid questions or claims, such heuristics provide limited signal, and we therefore omit this baseline to avoid introducing arbitrary or uninformative adaptations. Finally, (3) SingleObj-Acc, a single-objective RL baseline that optimizes accuracy-only.

Table 1 summarizes the task-level results. While absolute task performance remains constrained by retriever and generator capacity, our focus is on analyzing adaptive trade-off behavior under realistic system bottlenecks. Table 2–4 and Figure 2 further analyze AMORO-RAG from a weighting perspective, including the distribution of learned reward weights and their correlations with query characteristics, which constitute the core empirical evidence of our approach.

All systems share the same retrieval and generation backbone. We employ a hybrid retriever that combines dense retrieval using an external FAISS index [15] with BM25 sparse retrieval [26] over the same corpus, fused via Reciprocal Rank Fusion (RRF) [5]. An optional cross-encoder reranker is applied before generation. Answer generation is performed using a 7B instruction-tuned LLM [35],

Table 2: WeightNet statistics of query-conditioned reward weights. Std > 0 indicates non-trivial per-query adaptivity; Min/Max summarize the observed range.

Dataset	Statistic	w_{acc}	w_{faith}	w_{cost}	w_{lat}
HotpotQA	Mean	0.231	0.292	0.246	0.231
	Std	0.00082	0.00292	0.00139	0.00096
	Min	0.227	0.280	0.242	0.227
	Max	0.234	0.297	0.251	0.235
NQ	Mean	0.227	0.294	0.244	0.234
	Std	0.00210	0.00330	0.00200	0.00070
	Min	0.225	0.281	0.240	0.231
	Max	0.233	0.298	0.252	0.235
FEVER	Mean	0.252	0.317	0.268	0.163
	Std	0.00145	0.00370	0.00216	0.00018
	Min	0.2466	0.3020	0.2653	0.1624
	Max	0.2568	0.3237	0.2778	0.1639

Table 3: Average reward weights grouped by query length. Query length groups (Short/Medium/Long) are defined by dataset-specific terciles based on the empirical query length distribution. Across all datasets, longer queries are associated with higher faithfulness weights and lower cost/latency weights, indicating instance-conditioned trade-off adaptation.

Dataset	Query Length	w_{acc}	w_{faith}	w_{cost}	w_{lat}
HotpotQA	Short	0.231	0.290	0.247	0.232
	Medium	0.230	0.293	0.245	0.231
	Long	0.230	0.294	0.245↓	0.230↓
NQ	Short	0.227	0.294	0.245	0.235
	Medium	0.227	0.294	0.245	0.234
	Long	0.227	0.296	0.244↓	0.234↓
FEVER	Short	0.254	0.312	0.271	0.164
	Medium	0.251	0.318	0.268	0.163
	Long	0.250	0.320	0.267	0.163

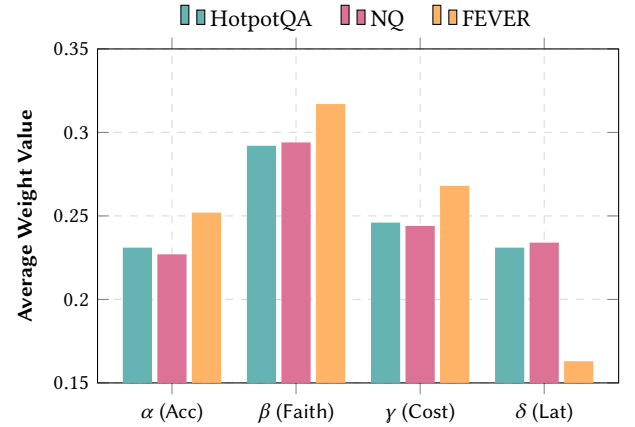
Table 4: Correlation between query length and reward weights.

Dataset	α	β	γ	δ
HotpotQA	-0.560	+0.764	-0.618	-0.958
NQ	+0.089	+0.233	-0.284	-0.550
FEVER	-0.704	+0.741	-0.716	-0.943

and reinforcement learning is conducted with PPO to optimize both the policy and the WeightNet, which predicts instance-conditioned reward weights $\lambda(x)$.

4.2 Task Performance and Efficiency

Table 1 reports task performance and efficiency across datasets. Overall, AMORO-RAG does not necessarily maximize any single

**Figure 2: Average reward weights learned by WeightNet across datasets. Dataset-level preferences emerge naturally: HotpotQA and NQ emphasize faithfulness, whereas FEVER assigns relatively low weight to latency.**

task metric, which is expected, as the goal of our approach is to learn instance-conditioned trade-offs rather than to optimize individual objectives in isolation. While accuracy-focused baselines can achieve higher task accuracy in some cases (e.g., on HotpotQA), they consistently incur substantially higher cost and latency. In contrast, AMORO-RAG exhibits a more balanced behavior, achieving competitive task performance while significantly reducing computational cost and response time.

HotpotQA. On the multi-hop benchmark, AMORO-RAG achieves comparable accuracy to Fixed-Weight while reducing token usage and slightly improving latency. In contrast, SingleObj-Acc yields the highest accuracy but shows clear efficiency degradation, indicating that optimizing accuracy alone can encourage over-generation or excessive computation in multi-hop settings.

Natural Questions. On NQ, AMORO-RAG matches Fixed-Weight in accuracy while achieving markedly better efficiency. The accuracy-only baseline again increases cost and latency substantially without consistent gains in accuracy.

FEVER. On FEVER, AMORO-RAG attains competitive label accuracy while achieving the best efficiency among compared methods. Similar to other datasets, SingleObj-Acc attains the highest accuracy but suffers from large increases in cost and latency, suggesting that a single-metric objective can lead to inefficient policies even for a classification-style task.

We note that the absolute values of the aggregate scalar reward are not directly comparable across systems, since each method optimizes different scalarizations and weight schedules. Therefore, we focus on the common metric space (accuracy, faithfulness proxy, cost, latency) and on evidence of instance-level adaptation.

4.3 WeightNet Behavior and Instance-Conditioned Adaptation

Table 2 and Figure 2 summarize dataset-level WeightNet behavior. Across all datasets, WeightNet assigns the largest average weight to faithfulness, reflecting the importance of evidence-grounded generation, while adjusting the relative emphasis on accuracy and efficiency depending on the dataset.

Dataset-level priors. HotpotQA exhibits a faithfulness-centered prior with relatively balanced weights on accuracy, cost, and latency, consistent with the need for multi-hop evidence chaining. NQ shows a similar but slightly more balanced distribution. FEVER displays a distinct pattern, increasing the weights on faithfulness and cost while assigning a notably lower weight to latency, aligning with a setting where short outputs make time less critical than evidence consistency and computation.

Evidence of per-query adaptation. Importantly, Table 2 shows non-zero standard deviations across all reward dimensions, confirming that WeightNet does not collapse to a constant policy. Although the absolute magnitude of variation is small due to softmax normalization and PPO stability constraints, these variations are systematic rather than random, and are further supported by strong correlations with query properties.

4.4 Query-Length Conditioning and Operating-Point Shifts

To probe instance-conditioned scalarization more directly, we analyze how learned weights vary with query length and how these shifts align with observed behavior.

Table 3 provides a coarse-grained view of this effect by grouping queries into length buckets (Short/Medium/Long) and reporting the average learned weights in each group. Across datasets, we observe a consistent operating-point shift: longer queries receive higher faithfulness weights, while cost and latency weights tend to decrease. Although the absolute changes are modest, the directionality is stable across benchmarks, supporting the interpretation that WeightNet encodes structured, query-dependent preferences rather than collapsing to a constant weighting.

Table 4 reports the correlation between query length and reward weights. Across datasets, we observe a consistent trend: longer queries are associated with higher faithfulness weights and lower cost/latency weights, implying that WeightNet learns to allocate more emphasis to evidence grounding and to penalize inefficiency more strongly (or equivalently, to select different trade-offs) depending on query characteristics.

The trend is most pronounced on HotpotQA ($\text{corr}(\text{len}, w_{\text{faith}}) = +0.764$; $\text{corr}(\text{len}, w_{\text{lat}}) = -0.958$), consistent with the multi-hop nature of the benchmark where query length more strongly reflects reasoning complexity. NQ shows a weaker but still consistent signal ($\text{corr}(\text{len}, w_{\text{faith}}) = +0.233$; $\text{corr}(\text{len}, w_{\text{lat}}) = -0.550$), reflecting that many NQ questions are short factoids. FEVER again exhibits strong correlations ($\text{corr}(\text{len}, w_{\text{faith}}) = +0.741$; $\text{corr}(\text{len}, w_{\text{lat}}) = -0.943$), matching the interpretation that longer claims demand stronger evidence alignment.

Taken together, these analyses support the central claim of AMORO-RAG: the model learns not only a global trade-off but also instance-conditioned operating points that systematically vary with query properties.

5 Discussion

5.1 What AMORO-RAG Learns (and What It Does Not)

Our results highlight a key distinction between optimizing a single metric and learning to trade off multiple objectives per instance. Across datasets, AMORO-RAG tends to preserve task performance while reducing cost and latency relative to static baselines. Meanwhile, accuracy-only optimization can yield higher peak accuracy, but tends to produce policies with higher token usage and longer end-to-end latency.

Crucially, the WeightNet analyses indicate structured adaptation: dataset-level priors differ (e.g., FEVER assigns substantially lower weight to latency than HotpotQA/NQ), and query-level weights correlate with query length, especially on multi-hop settings. This suggests that WeightNet captures both global task characteristics and instance-specific requirements, which static weighting cannot represent.

At the same time, we emphasize that the present study does not claim that instance-conditioned weighting alone resolves fundamental limitations of the underlying RAG pipeline. In particular, the absolute task performance (e.g., F1 on HotpotQA/NQ) remains low across systems, suggesting bottlenecks in retrieval or generation quality. In such regimes, improved scalarization mainly manifests as better efficiency control and more interpretable trade-off behavior, rather than large gains in accuracy.

5.2 Why Adaptive Trade-off Matters Under Low Absolute Performance

A potential limitation of our study is that absolute task accuracy remains modest, which may reduce the apparent impact of adaptation when viewed solely through aggregate performance metrics. However, much of the prior work on reinforcement learning for RAG systems has focused on optimizing fixed or globally defined objective trade-offs, implicitly assuming a single operating point across diverse queries.

In contrast, our work examines a complementary question: whether a RAG system can learn to adjust its reward preferences as a function of input characteristics, even when overall task performance is constrained.

From this perspective, the value of adaptation lies not in maximizing any single metric, but in revealing structured, instance-dependent optimization behavior that is not captured by static scalarization schemes.

We therefore position AMORO-RAG as an analysis-oriented framework that highlights how multi-objective preferences can vary across inputs. This perspective complements existing approaches that prioritize absolute performance improvements, and may inform future work on adaptive control mechanisms in retrieval-augmented generation.

6 Conclusion

We introduced AMORO-RAG, a reinforcement learning framework for retrieval-augmented generation that learns instance-conditioned reward scalarization over accuracy, faithfulness, cost, and latency. Rather than assuming a fixed or globally defined trade-off, as is common in prior multi-objective RAG and RL-based approaches, AMORO-RAG models reward preferences as query-dependent variables, predicted by a lightweight WeightNet and optimized jointly with the generator via reinforcement learning.

Across HotpotQA, Natural Questions, and FEVER, AMORO-RAG achieves competitive task performance while exhibiting more balanced efficiency profiles compared to static weighting schemes. While accuracy-focused optimization can attain higher peak accuracy in some settings, it is frequently associated with increased computational cost and latency; AMORO-RAG, by contrast, learns to modulate these trade-offs in a more measured manner. Importantly, the primary contribution of this work lies beyond aggregate task metrics. Analyses of WeightNet statistics, query-length-conditioned trends, and dataset-specific correlations provide consistent evidence that the learned policy selects different operating points across instances, reflecting structured, input-dependent optimization behavior.

Taken together, these findings demonstrate that instance conditioned scalarization offers a viable and interpretable mechanism for studying adaptive multi-objective optimization in RAG systems. By explicitly modeling how reward preferences vary with query characteristics, AMORO-RAG complements existing approaches that focus on improving absolute task performance, and opens new directions for understanding and controlling trade-offs in retrieval-augmented generation.

Limitations

Despite its advantages, the proposed approach has several limitations.

First, overfitting of WeightNet is a potential concern. Since WeightNet learns a mapping between query features and reward weights, mismatches between training and test query distributions may lead to inappropriate weight adjustments. This risk is especially pronounced when training data are biased toward specific query lengths or structural patterns, potentially resulting in oversimplified weighting behavior.

Second, the issue of conflicting objectives remains. Inherent trade-offs exist between accuracy and faithfulness, as well as between cost and latency, and for some queries, no weight combination may yield fully satisfactory outcomes [7]. In this work, we do not explicitly resolve such conflicts, but instead encourage WeightNet to learn reasonable trade-offs on average across queries.

Third, our formulation relies on linear scalarization to aggregate multiple objectives. While this choice offers interpretability and training stability, it cannot directly capture nonlinear interactions or hard constraints among objectives[27]. Addressing such requirements may require extensions based on constrained reinforcement learning [1] or hierarchical objective formulations.

In addition, our faithfulness measurements rely on dataset-specific proxy metrics, which limits direct comparability across benchmarks. Moreover, while we observe consistent instance-conditioned adaptation patterns (e.g., non-zero variance and stable correlations), the magnitude of learned weight variations remains relatively small. Finally, the effectiveness of adaptive scalarization is inherently constrained by the quality of the underlying retriever and generator, and improvements in these components are necessary to translate adaptive behavior into higher absolute task performance.

A Beyond One-Step Adaptive Weighting

In its current form, AMORO-RAG employs a one-step adaptive weighting strategy, in which the reward structure is determined through a single prediction of reward weights conditioned on the input query. While effective in capturing instance-level trade-offs, this formulation does not explicitly account for how a particular weight assignment influences downstream performance outcomes.

A promising direction for future work is to incorporate a policy feedback loop, in which the performance resulting from a given weight selection is fed back as input to WeightNet. Such a mechanism would allow the model to progressively refine reward preferences conditioned on query characteristics, based on observed optimization outcomes rather than a single-shot decision.

This extension would move beyond instance-level Pareto point selection toward instance-adaptive Pareto trajectory learning, enabling the model to learn how reward preferences should evolve over the course of optimization for a given query.

B Future Directions

This work opens several promising avenues for future research. First, evaluating how well the learned WeightNet generalizes to unseen domains or datasets would provide insight into the feasibility of domain-general reward adaptation. In particular, exploring richer conditioning signals or more expressive query representations may enable stronger and more granular instance-dependent reward adaptation. Second, incorporating query-level uncertainty estimation to enable risk-aware weighting strategies may further improve robustness and stability in RAG systems [16]. Third, extending the framework to a multi-agent reward decomposition setting—where retrievers, generators, and evaluators are treated as separate agents with learnable reward contributions—could broaden the applicability of RLHF-style optimization[23]. Finally, leveraging user feedback for continual online tuning in real deployment

environments represents an important direction for sustaining long-term performance [24].

References

- [1] Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. 2017. Constrained Policy Optimization. In *International Conference on Machine Learning (ICML)*.
- [2] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. In *International Conference on Learning Representations (ICLR)*.
- [3] Matthew Barker, Andrew Bell, Evan Thomas, James Carr, Thomas Andrews, and Umang Bhatt. 2025. Faster, Cheaper, Better: Multi-Objective Hyperparameter Optimization for LLM and RAG Systems. (2025). arXiv:2502.18635.
- [4] Lingjiao Chen, Matei Zaharia, and James Zou. 2024. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *Transactions on Machine Learning Research* (2024). <https://arxiv.org/abs/2305.05176>
- [5] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. 2009. Reciprocal Rank Fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 758–759.
- [6] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. 2024. Ragas: Automated Evaluation of Retrieval Augmented Generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*.
- [7] Tobias Falke, Leonardo F. R. Ribeiro, Belinda Fidalgo, Andre F. T. Martins, and Iryna Gurevych. 2019. Ranking Generated Summaries by Correctness: An Interesting but Challenging Application for Natural Language Inference. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*.
- [8] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. 2023. Retrieval-Augmented Generation for Large Language Models: A Survey. (2023). arXiv:2312.10997.
- [9] David Ha, Andrew Dai, and Quoc V. Le. 2017. Hypernetworks. In *International Conference on Learning Representations (ICLR)*.
- [10] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. 2018. Deep Reinforcement Learning That Matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
- [11] Sebastian Hofstätter, Jiecao Chen, Karthik Raman, and Hamed Zamani. 2023. FiD-Light: Efficient and Effective Retrieval-Augmented Text Generation. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*.
- [12] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C. Park. 2024. Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*.
- [13] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. 2022. Survey of Hallucination in Natural Language Generation. *Comput. Surveys* 55, 12 (2022), 1–38.
- [14] Zhengbao Jiang, Frank F. Xu, Luyu Gao, Zhiqiu Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Active Retrieval Augmented Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 7969–7992.
- [15] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547.
- [16] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, Scott Johnston, Sheer El-Showk, Andy Jones, Nelson Elhage, Tristan Hume, Anna Chen, Yuntao Bai, Kamal Ndousse, Lydia Cartwright, Amos Wu, Nova Evans, Wing Lian, Dario Amodei, Chris Olah, and Jared Kaplan. 2022. Language Models (Mostly) Know What They Know. *arXiv preprint arXiv:2207.05221* (2022).
- [17] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. 2019. Natural Questions: A Benchmark for Question Answering Research. *Transactions of the Association for Computational Linguistics (TACL)* 7 (2019), 452–466.
- [18] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33. 9459–9474.
- [19] Chunlin Liu and Xin Xu. 2015. Multiobjective Reinforcement Learning: A Comprehensive Overview. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 45, 3 (2015), 385–398.
- [20] Jinchang Luo, Mingquan Cheng, Fan Wan, Ni Li, Xiaoling Xia, Shuangshuang Tian, Tingcheng Bian, Haiwei Wang, Haohuan Fu, and Yan Tao. 2025. GlobalRAG: Enhancing Global Reasoning in Multi-hop Question Answering via Reinforcement Learning. (2025). arXiv:2510.20548.
- [21] Ningning Ma, Xiangyu Zhang, Hai-Tao Zheng, and Jian Sun. 2020. WeightNet: Revisiting the Design Space of Weight Networks. In *European Conference on Computer Vision (ECCV)*.
- [22] Aviv Navon, Aviv Shamsian, Gal Chechik, and Ethan Fetaya. 2021. Learning the Pareto Front with Hypernetworks. In *International Conference on Learning Representations (ICLR)*.
- [23] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 35. 27730–27744.
- [24] German I. Parisi, Ronald Kemker, Jose L. Part, Christopher Kanan, and Stefan Wermter. 2019. Continual Lifelong Learning with Neural Networks: A Review. *Neural Networks* 113 (2019).
- [25] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. In *Advances in Neural Information Processing Systems*.
- [26] Stephen Robertson and Hugo Zaragoza. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends® in Information Retrieval* 3, 4 (2009), 333–389.
- [27] Diederik M. Roijers, Peter Vamplew, Shimon Whiteson, and Richard Dazeley. 2013. A Survey of Multi-Objective Sequential Decision-Making. *Journal of Artificial Intelligence Research* 48 (2013), 67–113.
- [28] John Schulman, Sergey Levine, Philipp Moritz, Michael I. Jordan, and Pieter Abbeel. 2015. Trust Region Policy Optimization. In *Proceedings of the 32nd International Conference on Machine Learning*.
- [29] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347* (2017).
- [30] Emma Strubell, Ananya Ganesh, and Andrew McCallum. 2019. Energy and Policy Considerations for Deep Learning in NLP. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*.
- [31] James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. 2018. FEVER: a Large-scale Dataset for Fact Extraction and VERification. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*.
- [32] Hado van Hasselt, Arthur Guez, and David Silver. 2016. Learning Values Across Many Orders of Magnitude. In *Advances in Neural Information Processing Systems*.
- [33] Zeqiu Wu, Yushi Hu, Weijia Shi, Dzmitry Dziedziec, Morrow Mu, Percy Liang, and Hannaneh Hajishirzi. 2023. Fine-Grained Human Feedback Gives Better Rewards for Language Model Training. In *Advances in Neural Information Processing Systems*.
- [34] Canwen Xu, Yichong Xu, Shuohang Wang, Yang Liu, Chenguang Zhu, and Julian McAuley. 2023. Small Models are Valuable Plug-ins for Large Language Models. *arXiv preprint arXiv:2305.08848* (2023).
- [35] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, et al. 2024. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115* (2024).
- [36] Runzhe Yang, Xingyuan Sun, and Karthik Narasimhan. 2019. A Generalized Algorithm for Multi-Objective Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [37] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- [38] Hanning Zhang, Juntong Song, Juno Zhu, Yuanhao Wu, Tong Zhang, and Cheng Niu. 2025. RAG-Reward: Optimizing RAG with Reward Modeling and RLHF. (2025).